

CLOUD COMPUTING

LAB EXAM



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

LAB EXAM

Q1 – AWS IAM Setup Using AWS CLI and Console Verification (10 marks)

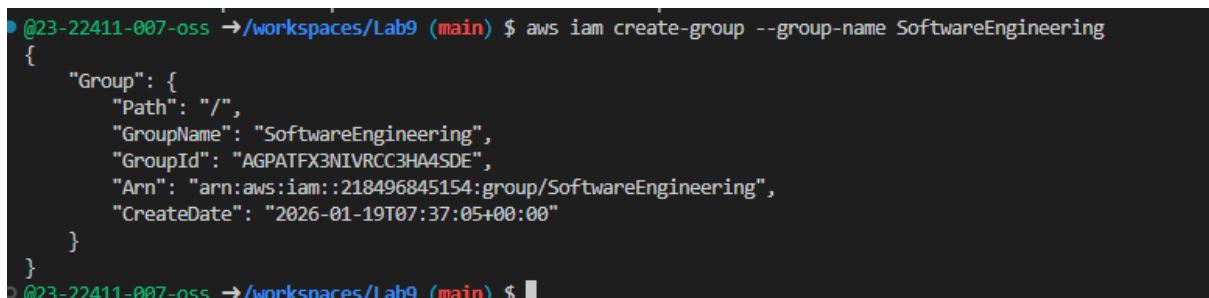
You are working as a junior cloud engineer. Your team needs an IAM setup for the **Software Engineering** group. Using the **AWS CLI** and then verifying in the **AWS Management Console**, complete the following.

You must:

- Use **your own name** for the IAM user (for example: Ali, Sara, YOURNAME).
- Use **exactly** this group name: SoftwareEngineering.
- Take the specified CLI and console screenshots at each point.

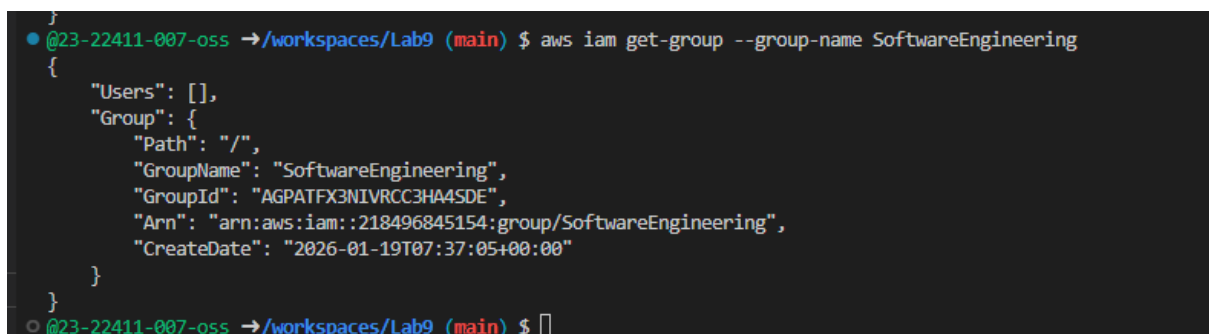
1. Create IAM group SoftwareEngineering using AWS CLI

- Use the AWS CLI to **create** an IAM group named SoftwareEngineering.



```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam create-group --group-name SoftwareEngineering
{
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPATFX3NIVRCC3HA4SDE",
    "Arn": "arn:aws:iam::218496845154:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:37:05+00:00"
  }
}
```

- **Save screenshot as: q1_create_group.png** — terminal showing the **create-group command** and its **output**.
- Then use the AWS CLI to **view the group details**.



```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPATFX3NIVRCC3HA4SDE",
    "Arn": "arn:aws:iam::218496845154:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:37:05+00:00"
  }
}
```

- **Save screenshot as: q1_group_details.png** — terminal showing output that includes:
 - The **group name** SoftwareEngineering, and
 - The **group ARN**.

2. Create IAM user (your name) and view details

- Using the AWS CLI, **create an IAM user** whose username is based on your own name (for example, Ali, Sara, YOURNAME).

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam create-user --user-name Amina
{
  "User": {
    "Path": "/",
    "UserName": "Amina",
    "UserId": "AIDATFX3NIVRAP2224JI7",
    "Arn": "arn:aws:iam::218496845154:user/Amina",
    "CreateDate": "2026-01-19T07:39:52+00:00"
  }
}
```

- Save screenshot as:** q1_create_user.png — terminal showing the **create-user** command and its output.
- Use the AWS CLI to **view the user details**.

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam get-user --user-name Amina
{
  "User": {
    "Path": "/",
    "UserName": "Amina",
    "UserId": "AIDATFX3NIVRAP2224JI7",
    "Arn": "arn:aws:iam::218496845154:user/Amina",
    "CreateDate": "2026-01-19T07:39:52+00:00"
  }
}
```

- Save screenshot as:** q1_user_details.png — terminal showing output that includes:
 - The **username**, and
 - The **user ARN**.

3. Add the IAM user to the SoftwareEngineering group

- Using the AWS CLI, **add the IAM user** (created above) to the SoftwareEngineering group.

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam add-user-to-group \
--user-name Amina \
--group-name SoftwareEngineering
```

- Save screenshot as:** q1_add_user_to_group.png — terminal showing the **add-user-to-group** command and its **output** (or lack of error).
- Use the AWS CLI to **view the group membership** and confirm your user is a member.

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Amina",
      "UserId": "AIDATFX3NIVRAP2224JI7",
      "Arn": "arn:aws:iam::218496845154:user/Amina",
      "CreateDate": "2026-01-19T07:39:52+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPATFX3NIVRCC3HA4SDE",
    "Arn": "arn:aws:iam::218496845154:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:37:05+00:00"
  }
}
@23-22411-007-oss →/workspaces/Lab9 (main) $
```

- **Save screenshot as:** q1_group_membership.png — terminal showing your IAM user listed as a **member** of SoftwareEngineering.

4. Attach AdministratorAccess managed policy to the SoftwareEngineering group

- Using the AWS CLI, **locate and identify** the AWS managed policy named AdministratorAccess, including its ARN (for example, by using `aws iam list-policies` or `aws iam get-policy` with the known ARN).

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam list-policies --scope AWS --query "Policies[?PolicyName=='AdministratorAccess']"
[
  {
    "PolicyName": "AdministratorAccess",
    "PolicyId": "ANPAIWMBCSKIEE64ZLYK",
    "Arn": "arn:aws:iam::aws:policy/AdministratorAccess",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 0,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2015-02-06T18:39:46+00:00",
    "UpdateDate": "2015-02-06T18:39:46+00:00"
  }
]
@23-22411-007-oss →/workspaces/Lab9 (main) $
```

- **Save screenshot as:** q1_find_admin_policy.png — terminal showing the command(s) and output that clearly include:
 - The **policy name** AdministratorAccess, and
 - The **policy ARN**.
- Using the AWS CLI, **attach** the AdministratorAccess managed policy to the SoftwareEngineering group using its policy ARN.

```
@23-22411-007-oss →/workspaces/Lab9 (main) $ aws iam attach-group-policy \
--group-name SoftwareEngineering \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
@23-22411-007-oss →/workspaces/Lab9 (main) $
```

- **Save screenshot as: q1_attach_admin_policy.png** — terminal showing the **attach-group-policy command** and confirmation (no error).

5. List attached policies of the SoftwareEngineering group

- Using the AWS CLI, **list the policies** attached to the SoftwareEngineering group (for example, list-attached-group-policies).

```

@23-22411-007-oss → /workspaces/Lab9 (main) $ aws iam list-attached-group-policies \
--group-name SoftwareEngineering
{
  "AttachedPolicies": [
    {
      "PolicyName": "AdministratorAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    }
  ]
}
@23-22411-007-oss → /workspaces/Lab9 (main) $

```

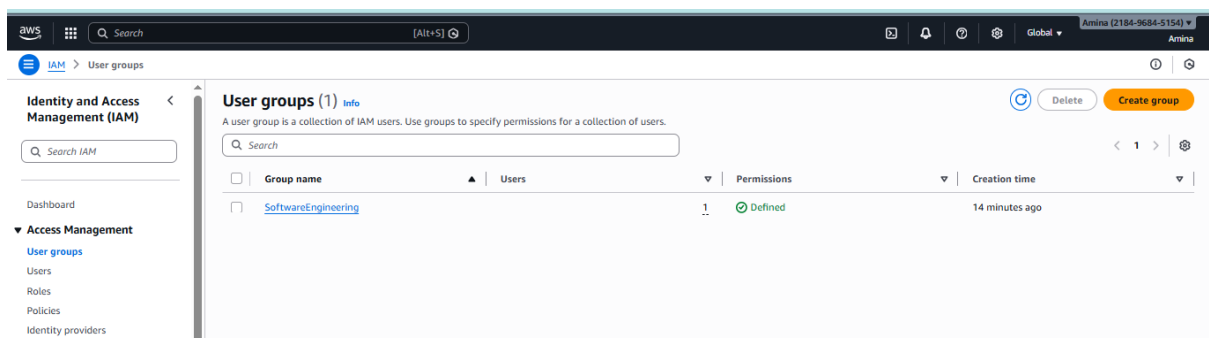
- **Save screenshot as: q1_list_group_policies.png** — terminal showing the output that clearly includes AdministratorAccess attached to SoftwareEngineering.

6. Verify IAM configuration in AWS Management Console

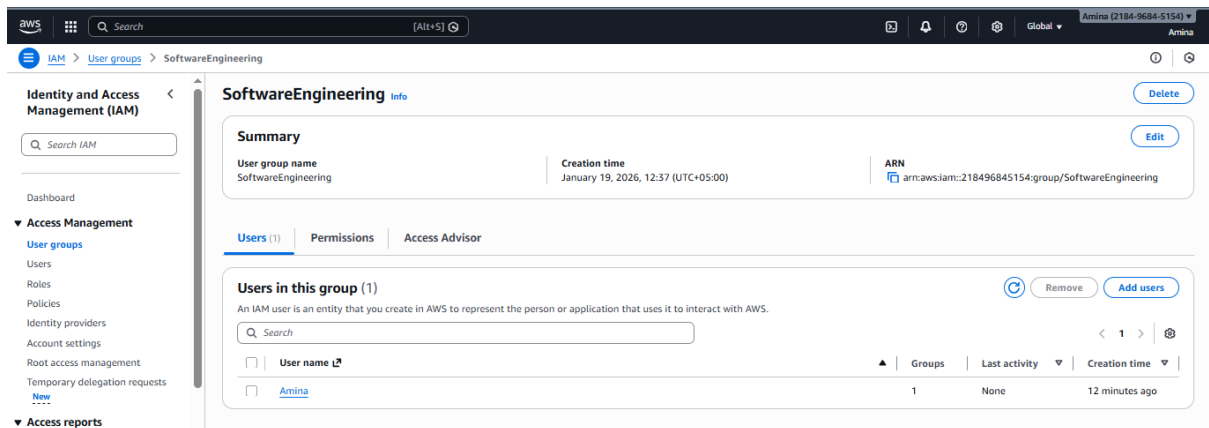
After completing all CLI steps above, log in to the **AWS Management Console** and verify that:

- The SoftwareEngineering group exists.
- Your IAM user exists and is part of that group.
- The group has the AdministratorAccess managed policy attached.

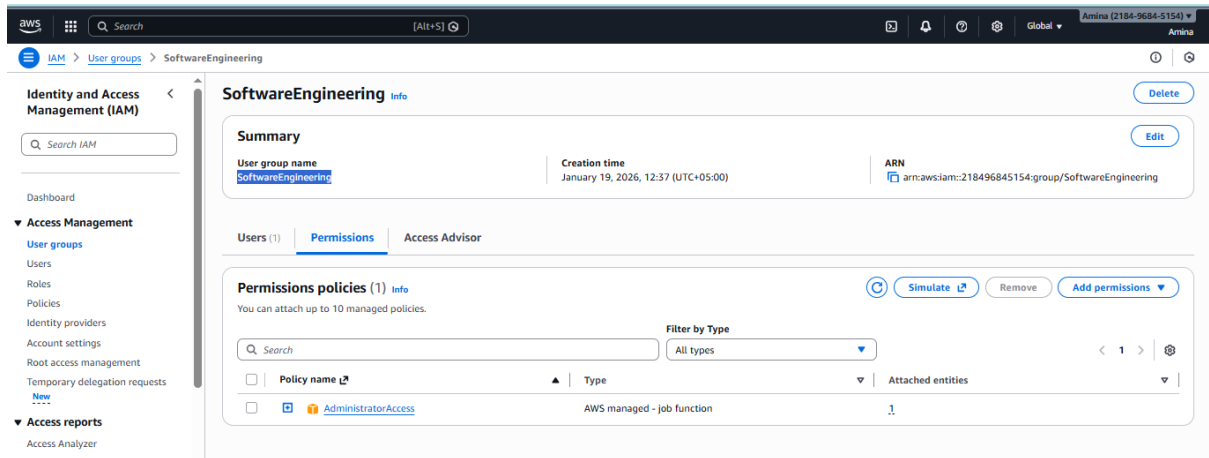
Take the following screenshots:



- **Save screenshot as: q1_console_group.png** — IAM console page clearly showing the **SoftwareEngineering group**.



- **Save screenshot as: q1_console_user_in_group.png** — IAM console page clearly showing your **IAM user** and that it belongs to the **SoftwareEngineering** group.



- **Save screenshot as: q1_console_group_policy.png** — IAM console page for the **SoftwareEngineering** group clearly showing the **AdministratorAccess** managed policy is attached.

Q2 – Terraform Lab: Simple AWS Environment with Nginx over HTTPS (30 marks)

You are working as an infrastructure engineer and must provision a simple application environment in AWS using **Terraform only**. No resources may be created manually in the AWS console.

You will create Terraform configuration and shell script files to deploy the required infrastructure.

You must:

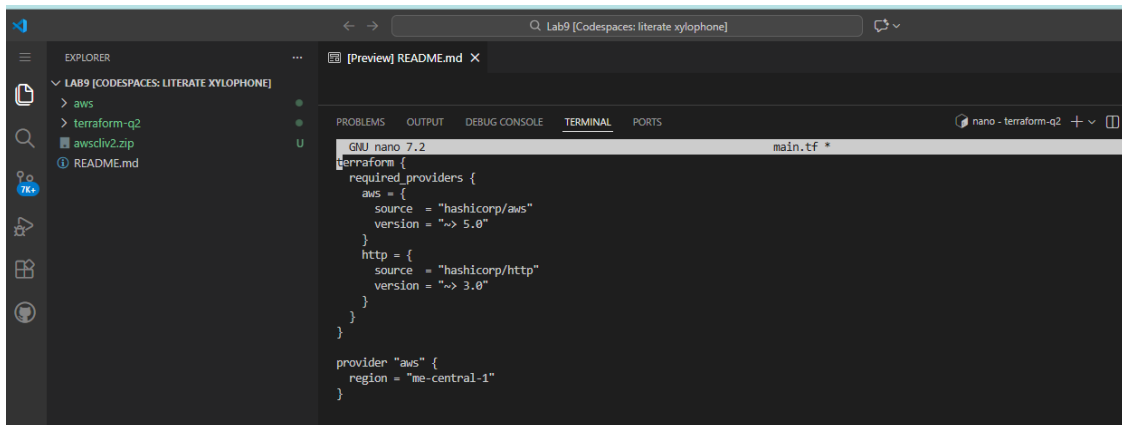
- Write all Terraform files and scripts yourself.
- Use a data "http" source and a locals block for your /32 IP.

1. Configure the AWS provider

Create the provider configuration so that the AWS provider reads from:

- ~/.aws/config
- ~/.aws/credentials

(You may use the default profile or a named profile.)



```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    http = {
      source = "hashicorp/http"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "me-central-1"
}

```

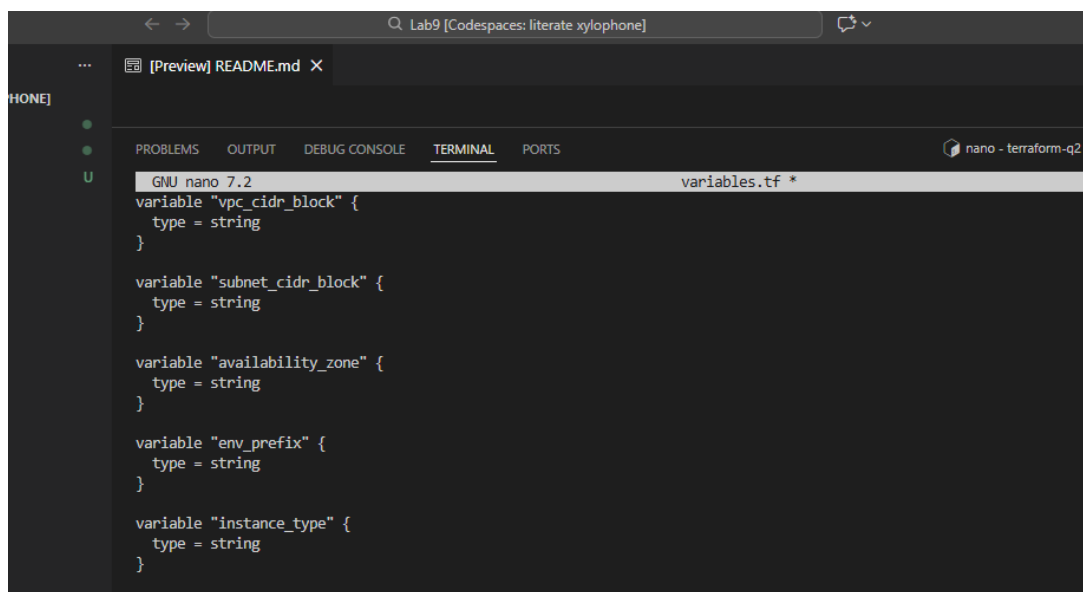
- **Save screenshot as:** q2_provider.png — editor or terminal view clearly showing the **provider** block in your .tf file (e.g., main.tf).

2. Define input variables

Define the following input variables:

- vpc_cidr_block
- subnet_cidr_block
- availability_zone
- env_prefix
- instance_type

Place them in variables.tf or any .tf file.



```

variable "vpc_cidr_block" {
  type = string
}

variable "subnet_cidr_block" {
  type = string
}

variable "availability_zone" {
  type = string
}

variable "env_prefix" {
  type = string
}

variable "instance_type" {
  type = string
}

```

- **Save screenshot as:** q2_variables.png — editor view showing all five variable definitions.

3. Create VPC and subnet

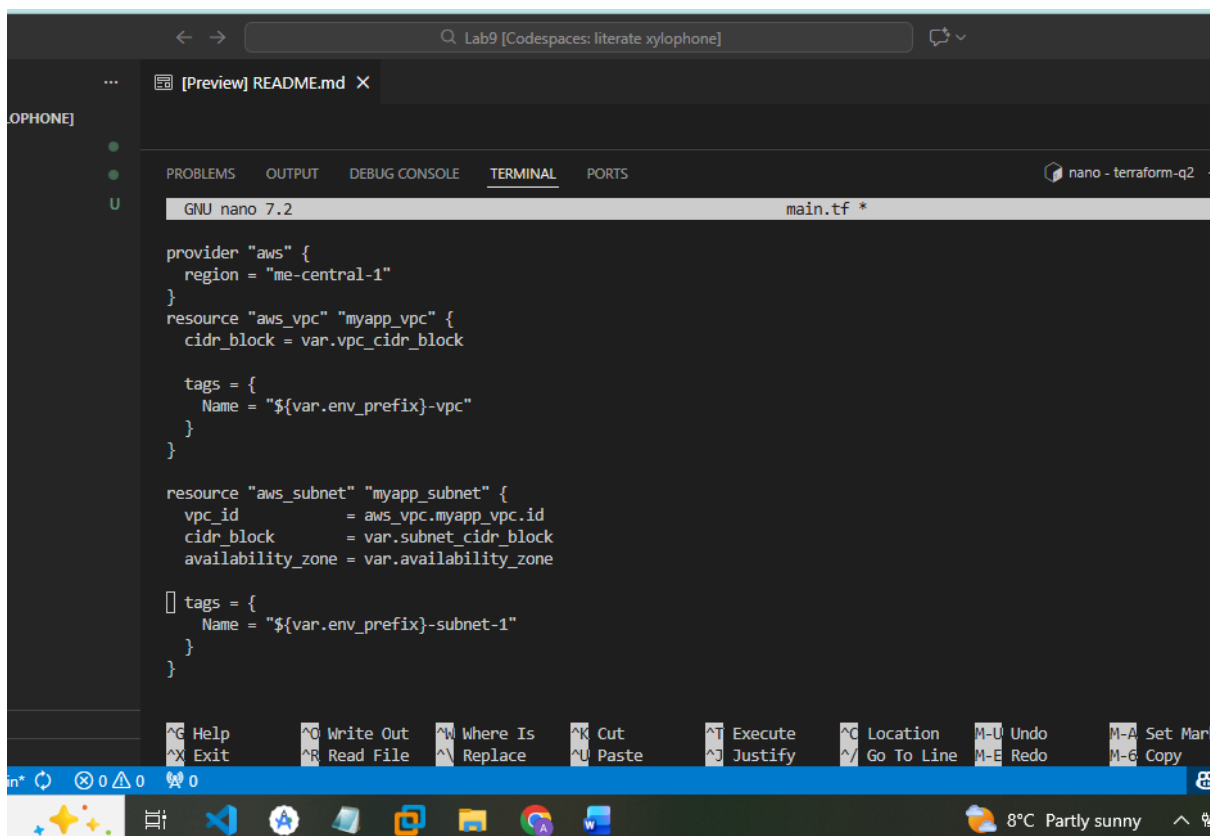
Using Terraform:

- Create a **VPC** (e.g., aws_vpc.myapp_vpc) using var.vpc_cidr_block as CIDR.
Tag:

Name = "<env_prefix>-vpc"

- Create a **subnet** attached to this VPC with:
 - var.subnet_cidr_block as CIDR,
 - var.availability_zone as AZ,
 - Tag:

Name = "<env_prefix>-subnet-1"



The screenshot shows a code editor window titled "Lab9 [Codespaces: literate xylophone]". The editor is displaying a Terraform configuration file named "main.tf". The configuration defines two resources: "aws_vpc" and "aws_subnet". The "aws_vpc" resource is named "myapp_vpc" and uses the variable "var.vpc_cidr_block" for its "cidr_block". It has a tag with the name "\${var.env_prefix}-vpc". The "aws_subnet" resource is named "myapp_subnet" and uses the "aws_vpc.myapp_vpc.id" for its "vpc_id", "var.subnet_cidr_block" for its "cidr_block", and "var.availability_zone" for its "availability_zone". It also has a tag with the name "\${var.env_prefix}-subnet-1". The editor interface includes a sidebar on the left with a file explorer, a top bar with tabs for "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL", and "PORTS", and a bottom status bar showing "8°C Partly sunny".

```

provider "aws" {
  region = "me-central-1"
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "myapp_subnet" {
  vpc_id            = aws_vpc.myapp_vpc.id
  cidr_block        = var.subnet_cidr_block
  availability_zone  = var.availability_zone

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

```

- **Save screenshot as:** q2_vpc_subnet.png — editor view showing VPC and subnet resources with tags.

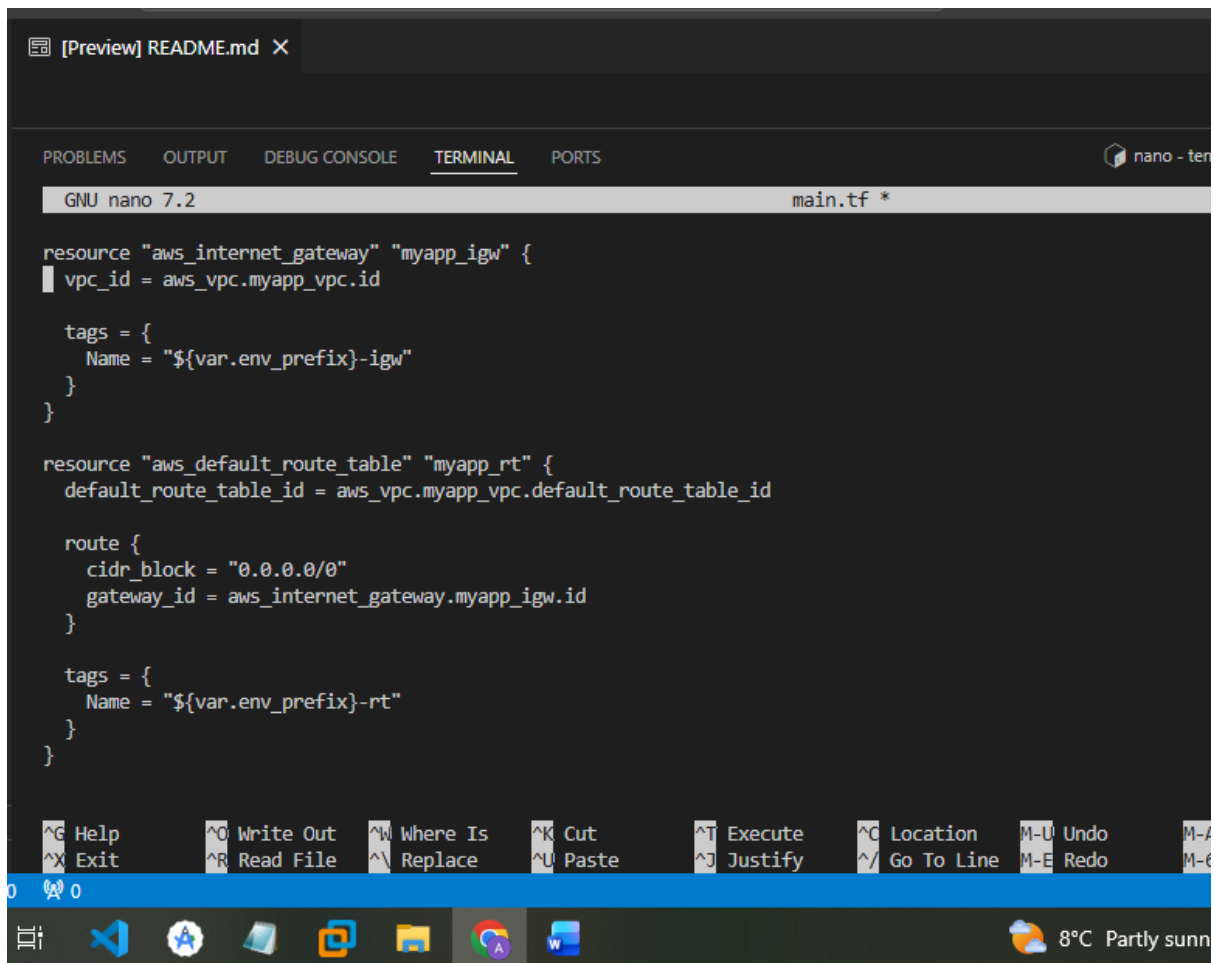
4. Create Internet Gateway and configure default route table

- Create an **Internet Gateway** attached to the VPC and tag it:

Name = "<env_prefix>-igw"

- Use the VPC's **default route table** to add a route 0.0.0.0/0 pointing to this IGW, and tag:

Name = "<env_prefix>-rt"



The screenshot shows a terminal window with the GNU nano 7.2 editor. The file being edited is main.tf. The configuration includes two resources: an aws_internet_gateway named 'myapp_igw' and an aws_default_route_table named 'myapp_rt'. The route table has a single route for 0.0.0.0/0 pointing to the internet gateway. Both resources are tagged with the name '{var.env_prefix}-igw' and '{var.env_prefix}-rt' respectively.

```

resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_default_route_table" "myapp_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

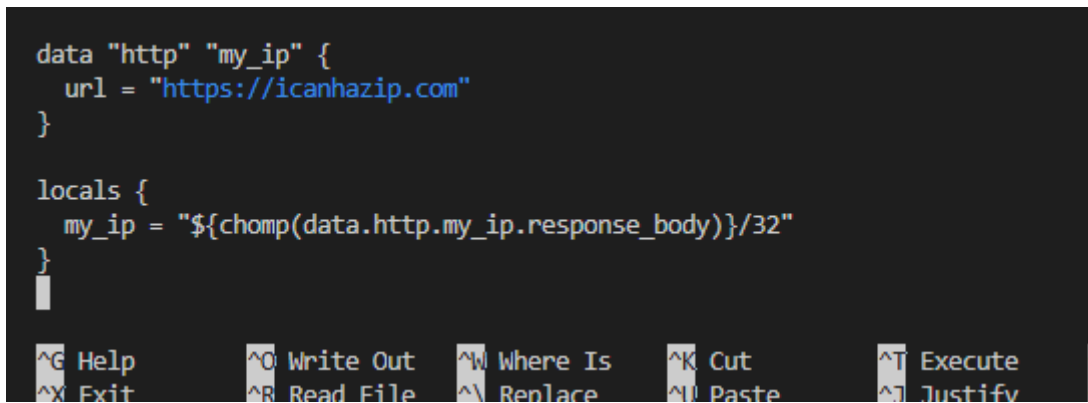
- **Save screenshot as:** q2_igw_route_table.png — editor view showing the IGW and default route table configuration, including the 0.0.0.0/0 route and tags.

5. Discover public IP and compute /32 CIDR using data + locals

- Use a Terraform data "http" block to query your **current public IP** (e.g., <https://icanhazip.com>).
- Use a locals block to:
 - Take the raw HTTP response body,
 - Strip trailing newline(s),
 - Append /32 to create a single-host CIDR locals.my_ip.

```
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"
}
```



- **Save screenshot as:** q2_http_and_locals.png — editor view showing the data "http" block and locals block with locals.my_ip.

6. Configure the default security group in the VPC

Manage the **default security group** of your VPC with:

- Ingress:
 - SSH (TCP 22) from locals.my_ip.
 - HTTP (TCP 80) from 0.0.0.0/0.
 - HTTPS (TCP 443) from 0.0.0.0/0.
- Egress:
 - All outbound traffic to 0.0.0.0/0.

Tag:

Name = "<env_prefix>-default-sg"

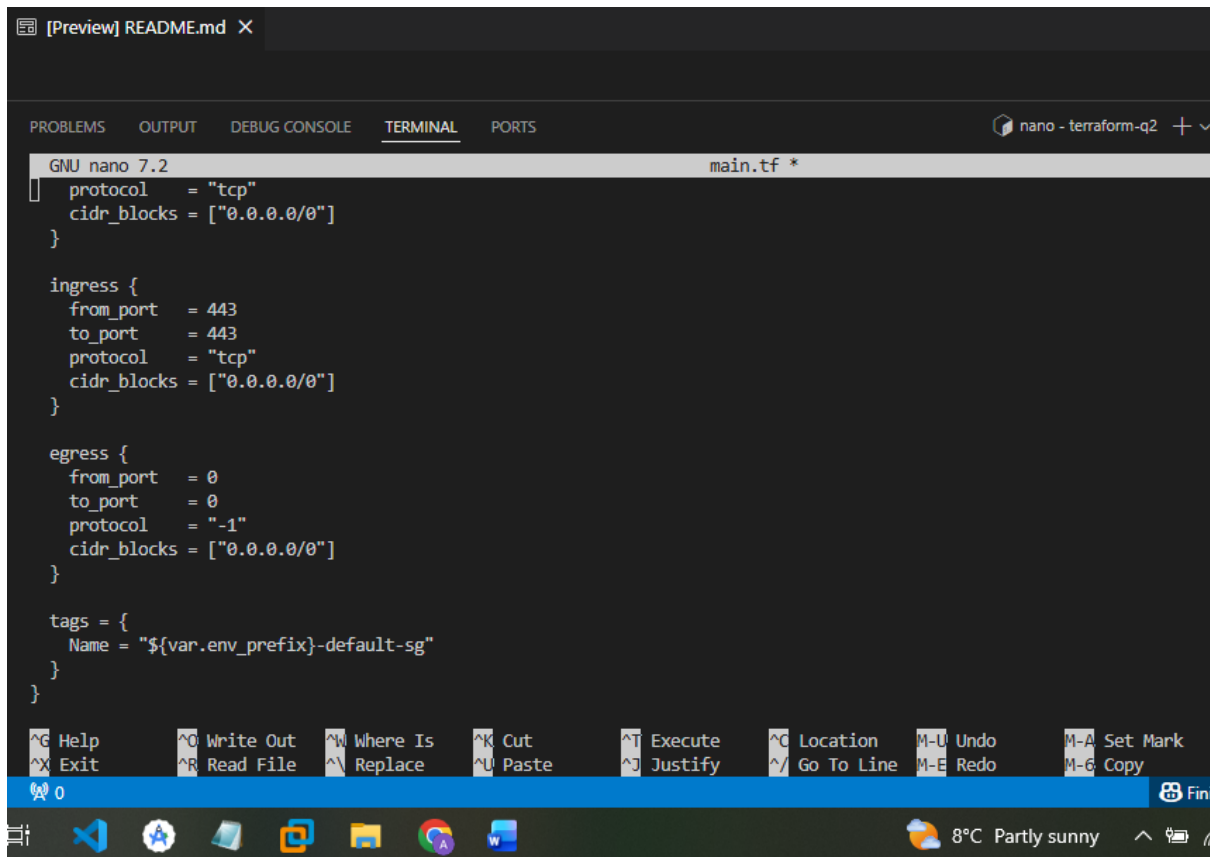
Code (security Group):

```
resource "aws_default_security_group" "default_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [locals.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
```

```
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
tags = {
    Name = "${var.env_prefix}-default-sg"
}
}
```



The screenshot shows a terminal window with the nano editor open, editing a file named `main.tf`. The configuration defines a default security group with the following settings:

```
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]

ingress {
  from_port = 443
  to_port = 443
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  from_port = 0
  to_port = 0
  protocol = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.env_prefix}-default-sg"
}
```

The terminal window includes a menu bar with options like Help, Write Out, Where Is, Cut, Execute, Location, Undo, Set Mark, Exit, Read File, Replace, Paste, Justify, Go To Line, Redo, and Copy. The status bar at the bottom shows the cursor at line 0 and the system tray with a temperature of 8°C and a weather forecast of 'Partly sunny'.

- **Save screenshot as:** `q2_default_sg.png` — editor view showing the default SG Terraform resource with all rules and tag.

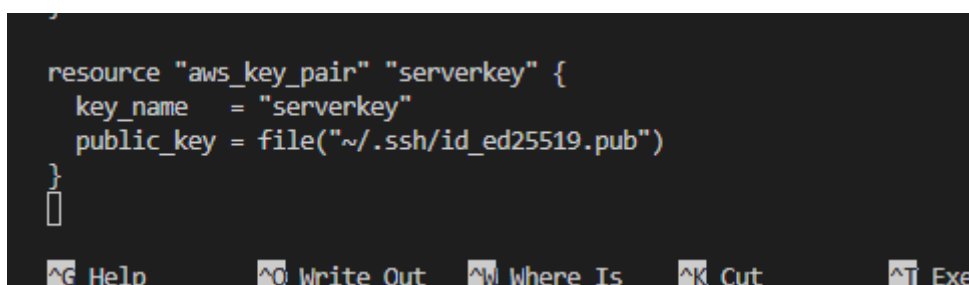
7. Create an AWS key pair for SSH

Create an `aws_key_pair` resource:

- Key name: `serverkey`
- Public key: `~/.ssh/id_ed25519.pub`

CODE:

```
resource "aws_key_pair" "serverkey" {
  key_name = "serverkey"
  public_key = file("~/ssh/id_ed25519.pub")
}
```



This is a close-up view of the Terraform code snippet from the previous block, showing the `aws_key_pair` resource definition in a dark-themed editor. The code is as follows:

```
resource "aws_key_pair" "serverkey" {
  key_name = "serverkey"
  public_key = file("~/ssh/id_ed25519.pub")
}
```

The bottom of the image shows the same menu bar as the first screenshot, with options like Help, Write Out, Where Is, Cut, and Execute visible.

- **Save screenshot as:** q2_keypair.png — editor view showing the key pair resource referencing ~/.ssh/id_ed25519.pub.

8. Create the EC2 instance resource

Launch an EC2 instance that:

- Uses an Amazon Linux 2023 AMI (hard-coded ID).
- Uses var.instance_type.
- Is in the subnet you created.
- Attaches the **default security group** from step 6.
- Uses var.availability_zone.
- Has a public IP address.
- Uses key pair serverkey.
- Uses user_data from a local file entry-script.sh.
- Is tagged:

Name = "<env_prefix>-ec2-instance"

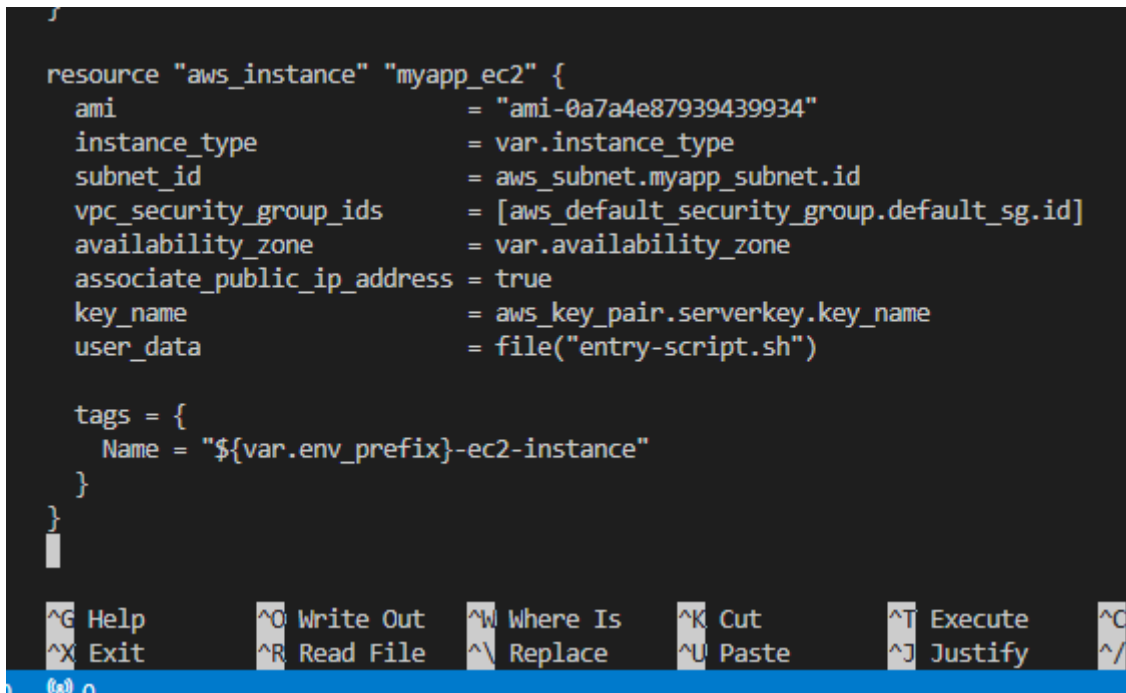
CODE:

```
resource "aws_instance" "myapp_ec2" {
  ami            = "ami-0a7a4e87939439934"
  instance_type  = var.instance_type
  subnet_id      = aws_subnet.myapp_subnet.id
  vpc_security_group_ids = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name        = aws_key_pair.serverkey.key_name
  user_data        = file("entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

```
resource "aws_instance" "myapp_ec2" {
  ami                = "ami-0a7a4e87939439934"
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.myapp_subnet.id
  vpc_security_group_ids = [aws_default_security_group.default_sg.id]
  availability_zone   = var.availability_zone
  associate_public_ip_address = true
  key_name            = aws_key_pair.serverkey.key_name
  user_data           = file("entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```



- **Save screenshot as:** q2_ec2_resource.png — editor view showing the EC2 instance resource with subnet, SG, key pair, user_data, and tag.

9. Create entry-script.sh to configure Nginx + HTTPS

In the same directory as your .tf files, create entry-script.sh that:

- Installs **Nginx**.
- Generates/configures a **self-signed TLS certificate**.
- Configures Nginx to:
 - Listen on **HTTPS (443)** using that certificate,
 - Serve **HTTP (80)** (redirect to HTTPS or serve a page).
- Enables Nginx on boot and ensures it is running after the script.
- Serves a page that includes:
 - Your **name**, and
 - The word **“Terraform”** (e.g. This is YOURNAME's Terraform environment.).

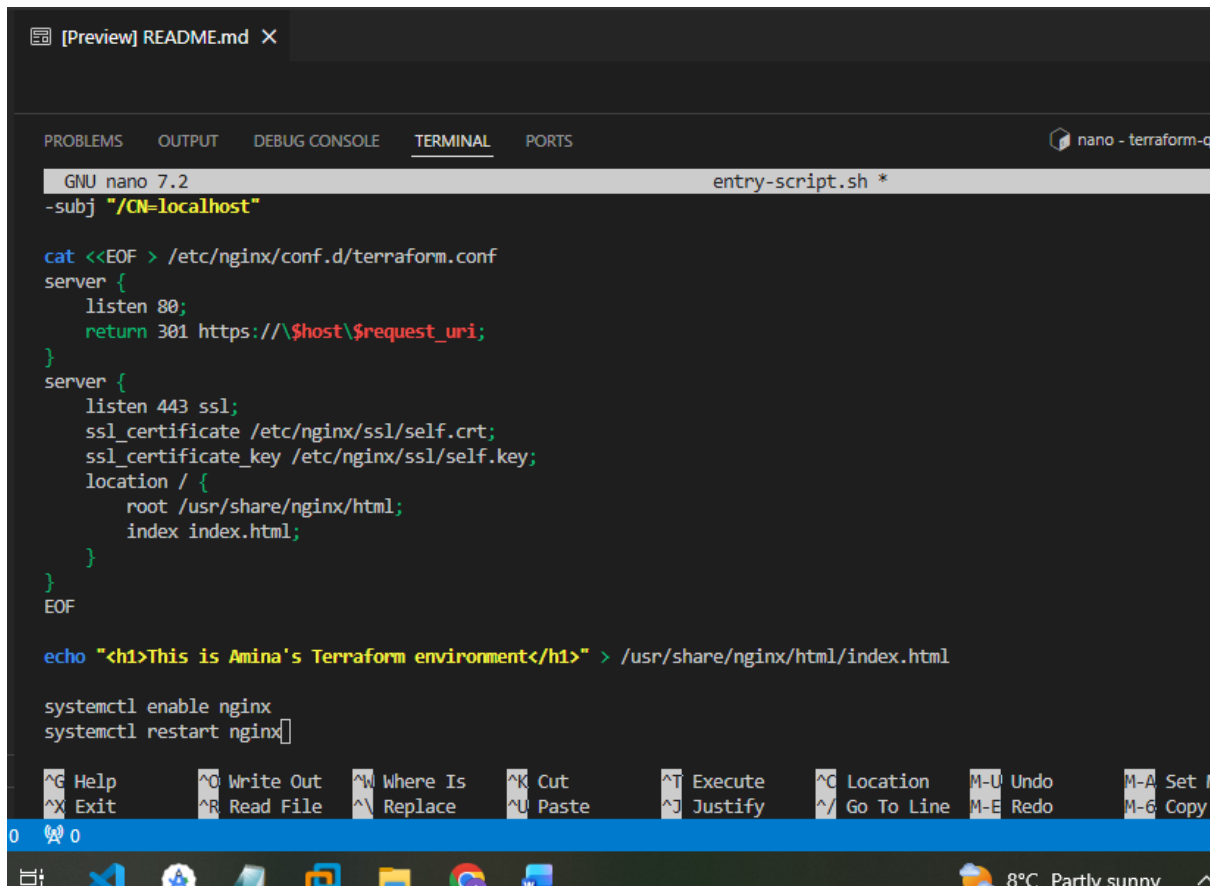
CODE:

```
#!/bin/bash

dnf install -y nginx openssl

mkdir -p /etc/nginx/ssl
```

```
openssl req -x509 -nodes -days 365 \  
-newkey rsa:2048 \  
-keyout /etc/nginx/ssl/self.key \  
-out /etc/nginx/ssl/self.crt \  
-subj "/CN=localhost"  
cat <<EOF > /etc/nginx/conf.d/terraform.conf  
server {  
    listen 80;  
    return 301 https://\${host}\$request_uri;  
}  
server {  
    listen 443 ssl;  
    ssl_certificate /etc/nginx/ssl/self.crt;  
    ssl_certificate_key /etc/nginx/ssl/self.key;  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
    }  
}  
EOF  
echo "<h1>This is Amina's Terraform environment</h1>" > /usr/share/nginx/html/index.html  
systemctl enable nginx  
systemctl restart nginx
```



```
[Preview] README.md X

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS nano - terraform-c

GNU nano 7.2 entry-script.sh *
-subj "/CN=localhost"

cat <<EOF > /etc/nginx/conf.d/terraform.conf
server {
    listen 80;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/self.crt;
    ssl_certificate_key /etc/nginx/ssl/self.key;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
EOF

echo "<h1>This is Amina's Terraform environment</h1>" > /usr/share/nginx/html/index.html

systemctl enable nginx
systemctl restart nginx
```

- **Save screenshot as:** q2_entry_script.png — editor or terminal view showing the full content of entry-script.sh.

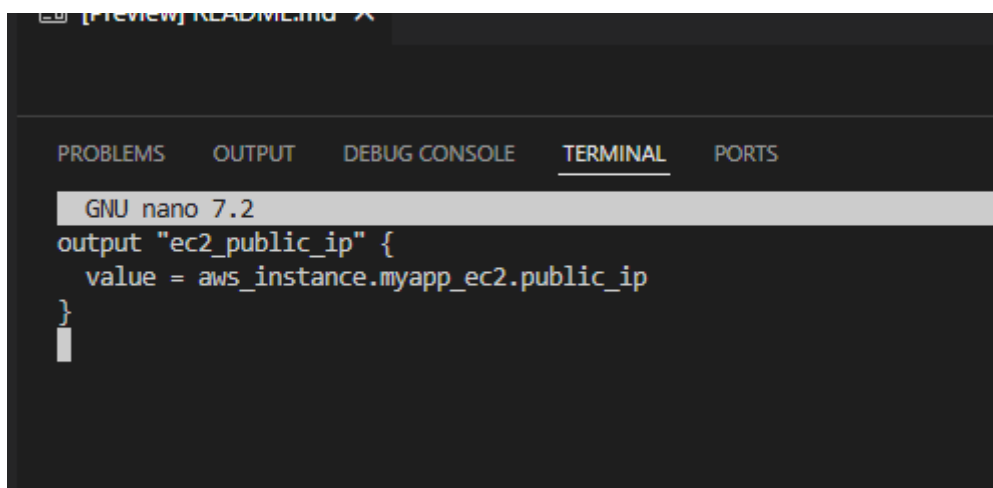
10. Add Terraform output for public IP

Add an output (e.g. ec2_public_ip) to print the instance's public IP after terraform apply.

```
output "ec2_public_ip" {
```

```
    value = aws_instance.myapp_ec2.public_ip
```

```
}
```



```
[Preview] README.md X

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

GNU nano 7.2
output "ec2_public_ip" {
    value = aws_instance.myapp_ec2.public_ip
}
```


- **Save screenshot as:** q2_output_block.png — editor view showing the Terraform output exposing the EC2 public IP.

11. Set variable values for apply time

When running terraform apply, use:

vpc_cidr_block = "10.0.0.0/16"

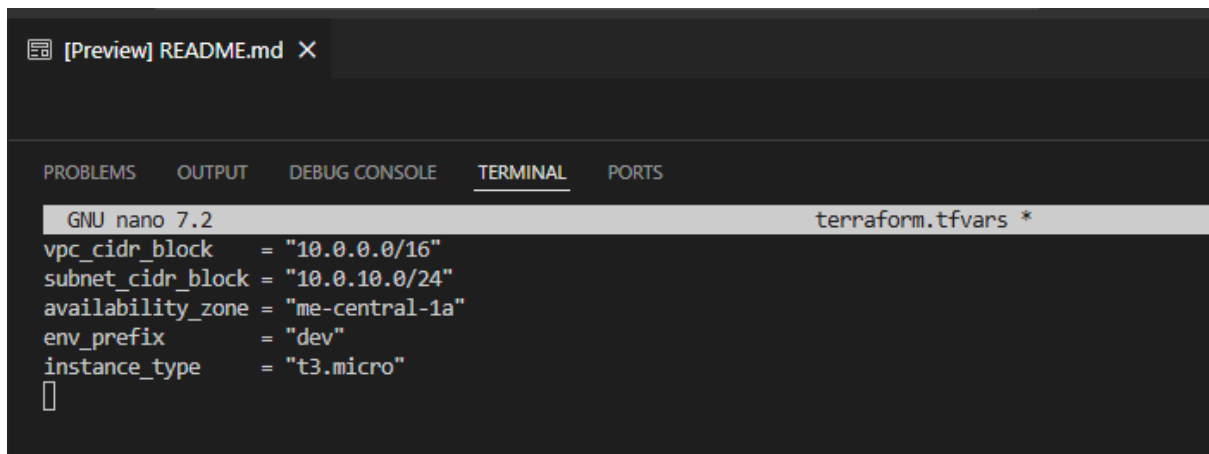
subnet_cidr_block = "10.0.10.0/24"

availability_zone = "me-central-1a"

env_prefix = "dev"

instance_type = "t3.micro"

(via terraform.tfvars, -var flags, or defaults)



The screenshot shows a terminal window with a dark background. At the top, there's a tab labeled "[Preview] README.md X". Below the tab bar, there are tabs for "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL" (which is selected), and "PORTS". The terminal content shows the GNU nano 7.2 editor editing the file terraform.tfvars *. The text in the terminal is as follows:

```
GNU nano 7.2 terraform.tfvars *
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix = "dev"
instance_type = "t3.micro"
█
```

- **Save screenshot as:** q2_tfvars_or_vars.png — editor view of terraform.tfvars or CLI evidence that these values are used.

12. Run Terraform commands and capture outputs

From the Terraform project directory:

terraform init

```
● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Finding hashicorp/http versions matching "~> 3.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
○ @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $
```

- **Save screenshot as:** q2_terraform_init.png — terminal showing terraform init and successful initialization.

terraform plan

```
[Preview] README.md X
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ nano main.tf
● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ ls ~/.ssh/id_ed25519.pub
/home/codespace/.ssh/id_ed25519.pub
● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform fmt
● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform validate
Success! The configuration is valid.

● @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform plan
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

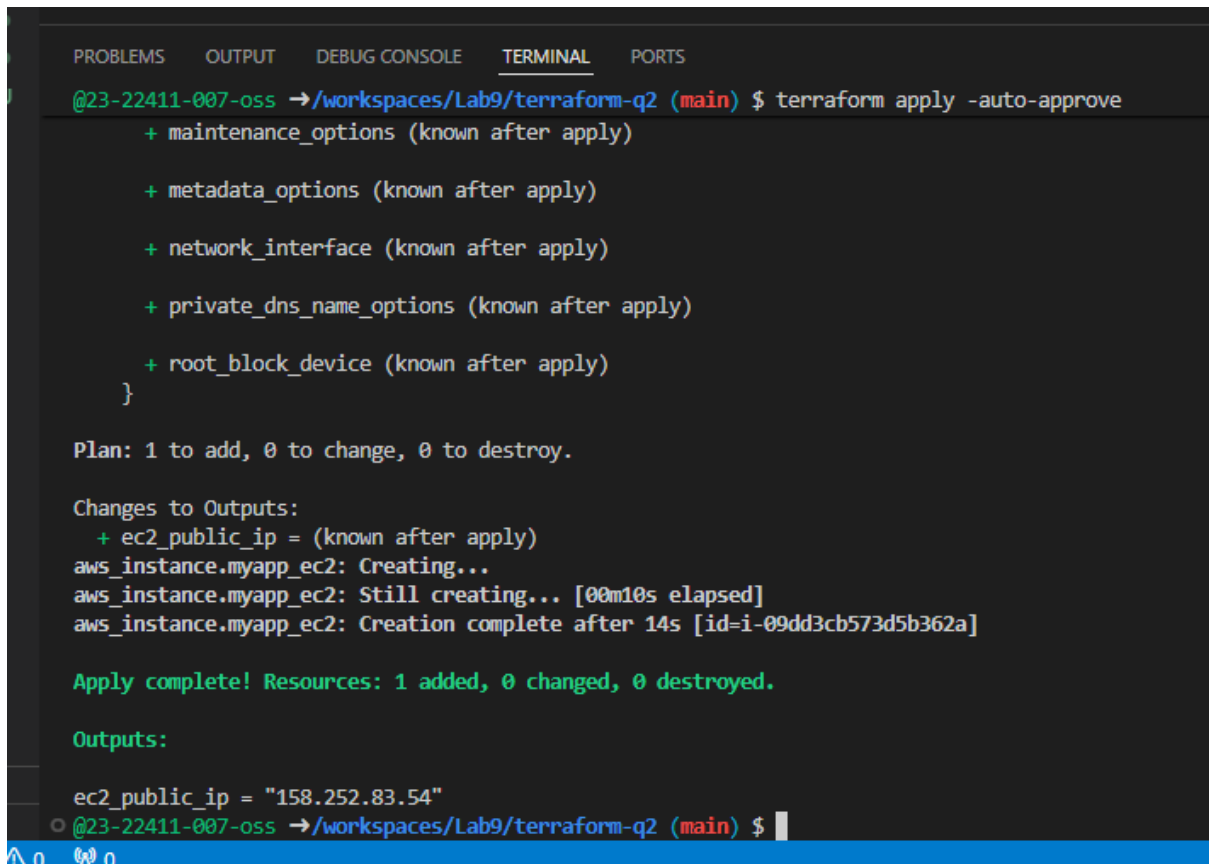
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols
+ create

Terraform will perform the following actions:

# aws_default_route_table.myapp_rt will be created
+ resource "aws_default_route_table" "myapp_rt" {
  + arn                = (known after apply)
  + default_route_table_id = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + route              = [
    + {
      + cidr_block = "0.0.0.0/0"
    }
  ]
}
```

- **Save screenshot as:** q2_terraform_plan.png — terminal showing terraform plan and part of the plan.

terraform apply

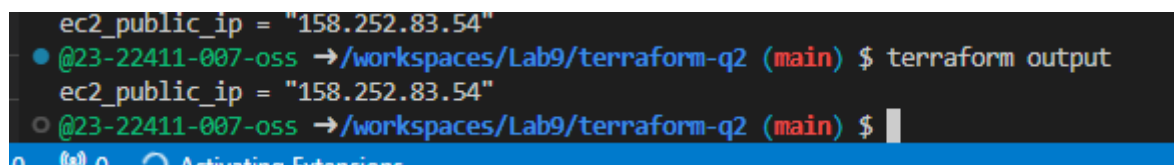
A screenshot of a terminal window with a dark background. The terminal shows the execution of the Terraform apply command. It lists resources to be added: maintenance_options, metadata_options, network_interface, private_dns_name_options, and root_block_device. It then shows the plan (1 to add, 0 to change, 0 to destroy) and the changes to outputs, specifically ec2_public_ip. The output shows the EC2 instance being created, still creating, and then creation complete after 14s. The final output is the public IP address 158.252.83.54. The terminal prompt is @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) \$.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform apply -auto-approve
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ ec2_public_ip = (known after apply)
aws_instance.myapp_ec2: Creating...
aws_instance.myapp_ec2: Still creating... [00m10s elapsed]
aws_instance.myapp_ec2: Creation complete after 14s [id=i-09dd3cb573d5b362a]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outputs:
ec2_public_ip = "158.252.83.54"
@23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $
```

(You may use -auto-approve.)

- **Save screenshot as:** q2_terraform_apply.png — terminal showing terraform apply and successful resource creation.

terraform output

A screenshot of a terminal window showing the output of the Terraform output command. It displays the public IP address of the EC2 instance: 158.252.83.54. The terminal prompt is @23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) \$.

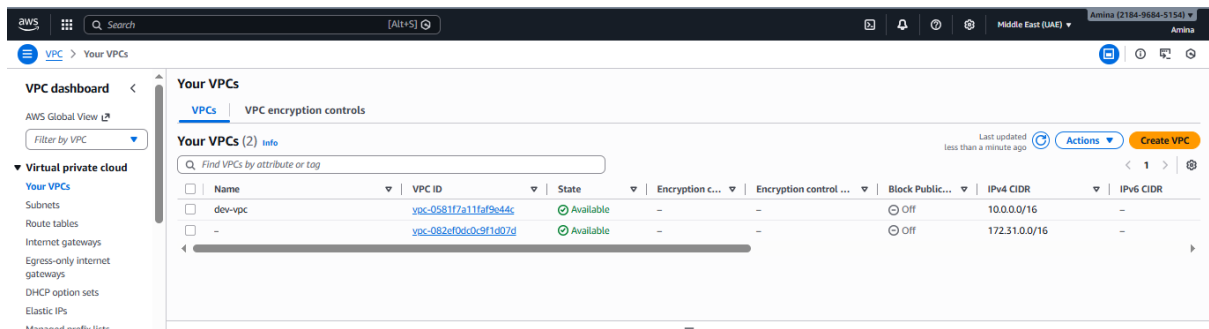
```
@23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $ terraform output
ec2_public_ip = "158.252.83.54"
@23-22411-007-oss → /workspaces/Lab9/terraform-q2 (main) $
```

- **Save screenshot as:** q2_terraform_output.png — terminal showing outputs including the EC2 public IP.

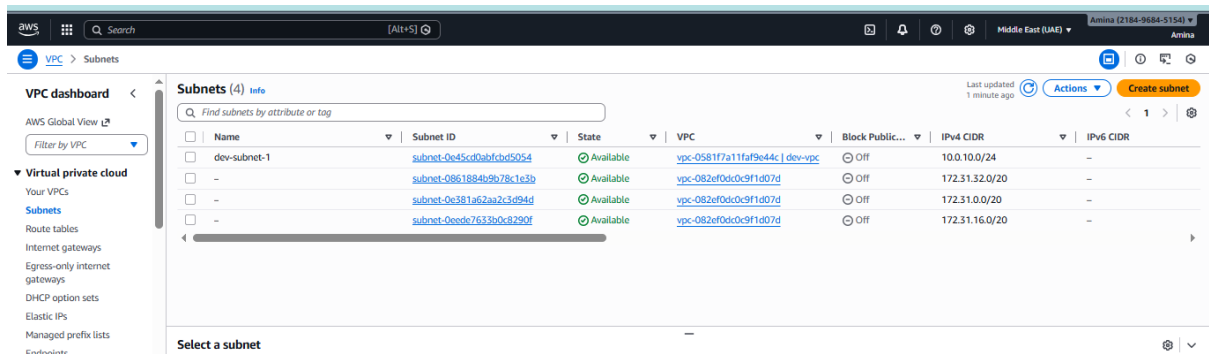
13. Verify Terraform resources in AWS console

Use the AWS console to confirm:

- **VPC and Subnet**

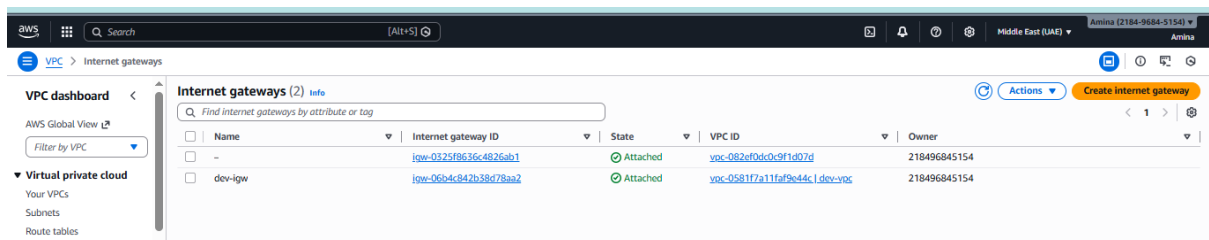


- **Save screenshot as: q2_console_vpc.png** — VPC with CIDR 10.0.0.0/16 and tag Name = dev-vpc.

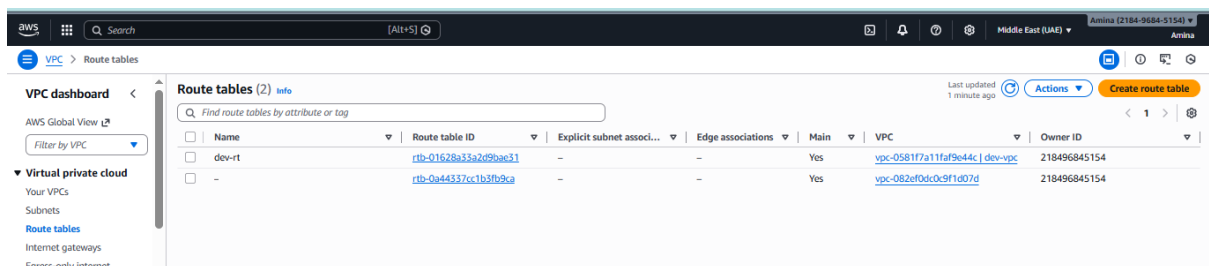


- **Save screenshot as: q2_console_subnet.png** — subnet with CIDR 10.0.10.0/24, AZ me-central-1a, tag Name = dev-subnet-1.

○ Internet Gateway and Route Table

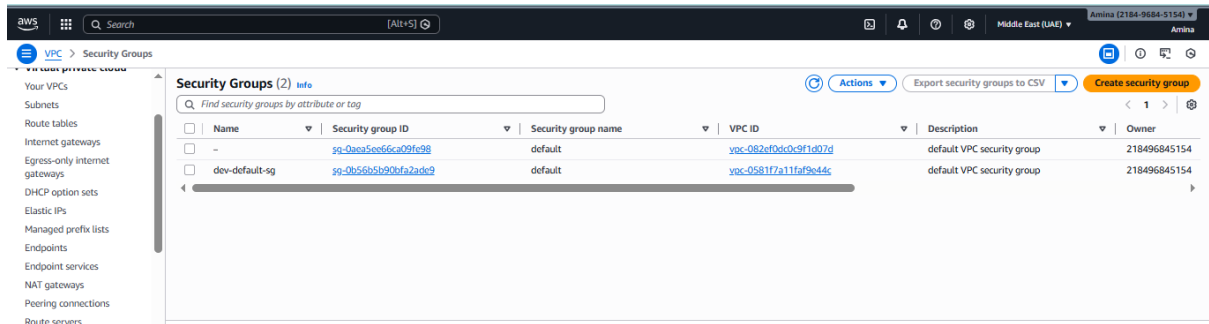


- **Save screenshot as: q2_console_igw.png** — IGW attached to the VPC with tag Name = dev-igw.



- **Save screenshot as: q2_console_route_table.png** — route table with tag Name = dev-rt and route 0.0.0.0/0 to the IGW.

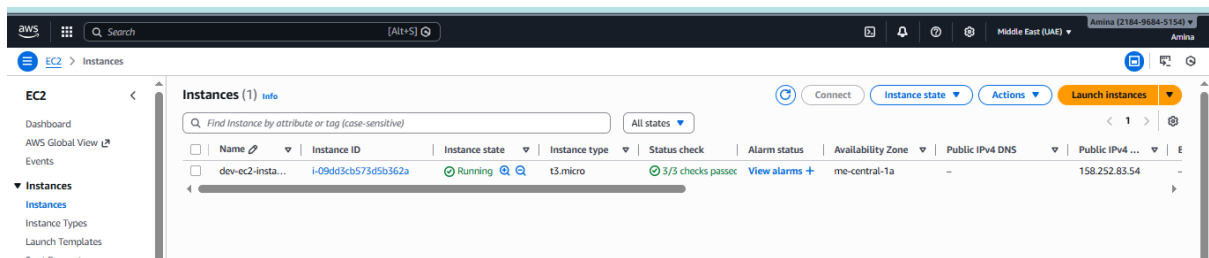
○ Security Group



▪ **Save screenshot as: q2_console_sg.png** — default SG showing:

- SSH (22) from your /32 host,
- HTTP (80) from 0.0.0.0/0,
- HTTPS (443) from 0.0.0.0/0,
- Egress all to 0.0.0.0/0,
- Tag Name = dev-default-sg.

○ **EC2 instance**



▪ **Save screenshot as: q2_console_ec2.png** — instance:

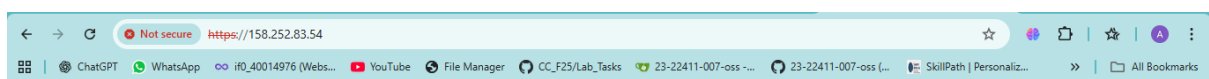
- In subnet 10.0.10.0/24, AZ me-central-1a,
- With a public IP,
- Using key pair serverkey,
- Tagged Name = dev-ec2-instance.

14. Verify HTTPS access from browser

From your local machine:

`https://<public-ip-of-instance>`

Accept any self-signed cert warning and verify the page content.



This is Amina's Terraform environment

- **Save screenshot as:** q2_https_browser.png — browser screenshot showing:
 - Address bar https://<public-ip-of-instance>,
 - Nginx page with your **name** and **“Terraform”**.

Q3 – Ansible Playbook for EC2 Web Server Using Q2 Instance (10 marks)

You are working as a DevOps engineer. A previous solution used a shell script on an Amazon Linux EC2 instance to:

- Update the system,
- Install and start Apache HTTPD,
- Fetch instance metadata (public IP and public hostname) using **IMDSv2**,
- Print the public IP,
- Restart the HTTPD service.

Your task is to convert this logic into an **Ansible-based** solution targeting the EC2 instance created in **Step 2 (Q2)**.

If Nginx is running from Q2’s user data, you must **stop/disable or uninstall nginx** so that httpd can bind to port 80.

1. Create Ansible inventory file hosts

In a directory such as Lab_exam/ansible/, create hosts that:

- Defines group ec2 with your Q2 instance’s **public IP**.
- Sets group vars:
 - ansible_user = ec2-user
 - ansible_ssh_private_key_file = ~/.ssh/id_ed25519
 - ansible_ssh_common_args to disable strict host key checking (-o StrictHostKeyChecking=no).

```

GNU nano 7.2 hosts *
[ec2]
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAITawasj3U+Alwui6lNqOythRMsNLftRnm154nmfrxgUf labexam
[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
  
```

- **Save screenshot as:** q3_hosts.png — editor or terminal view showing the complete hosts file (group, IP, vars).

2. Create project-level ansible.cfg

In the same directory, create `ansible.cfg` that:

- Disables host key checking.
- Sets remote Python interpreter to `/usr/bin/python3`.
- (Optional) Sets inventory = `./hosts`.

- **Save screenshot as:** `q3_ansible_cfg.png` — editor view showing `ansible.cfg` with required settings.

3. Create Ansible playbook (e.g. `my-playbook.yml`)

Create `my-playbook.yml` that:

- Targets group `ec2`.
- Uses `become: true`.
- Performs actions equivalent to the shell script:
 - Update system packages.
 - Stop/disable or uninstall `nginx` if present (to free port 80).
 - Install `httpd`.
 - Start and enable `httpd` service.
 - Get IMDSv2 token using uri:
 - URL: `http://169.254.169.254/latest/api/token`
 - Method: `PUT`
 - Header: `X-aws-ec2-metadata-token-ttl-seconds: 21600`
 - Use token to fetch:
 - `http://169.254.169.254/latest/meta-data/public-ipv4`
 - `http://169.254.169.254/latest/meta-data/public-hostname`
 - Print **public IP** with `debug`.
 - Restart `httpd`.

- **Save screenshot as:** `q3_playbook.png` — editor view showing the full contents of `my-playbook.yml`.

4. Run the Ansible playbook

Execute:

ansible-playbook -i hosts my-playbook.yml

(or without -i if your ansible.cfg sets the inventory.)

- **Save screenshot as:** q3_play_run.png — terminal showing:
 - The ansible-playbook command,
 - All tasks (update, nginx handling, httpd, IMDSv2, debug of public IP),
 - Successful completion.

5. Verify HTTP access

From your local machine:

http://<public-ip-ec2>

Confirm Apache (httpd) is responding.

- **Save screenshot as (optional):** q3_http_browser.png — browser or curl output showing HTTP access to http://<public-ip-ec2>.

Files Required (Q3) in Lab_exam repo, e.g. Lab_exam/ansible/:

- hosts
- ansible.cfg
- my-playbook.yml

Cleanup (ungraded)

After collecting all evidence:

1. From your Terraform project directory (Q2):

terraform destroy -auto-approve

- **Save screenshot as (recommended):** cleanup_terraform_destroy.png — terminal showing destroy output.

2. In AWS console, verify that no lab-related EC2 instances remain running.

- **Save screenshot as (recommended):** cleanup_ec2_console.png — EC2 console showing no remaining lab instances.

(If your instructor gives different cleanup instructions, follow those.)